

基于 CIFAR-10 不平衡二分类任务的五折交叉验证学习型实验报告

本实验以 CIFAR-10 中的 Dog 与 Non-Dog 图像为对象，构造正负样本比例为 1:4 的不平衡二分类任务，学习并比较两种模型评估协议：重复分层五折交叉验证（Repeated Stratified 5-Fold Cross-Validation，简称 CV）和单次分层随机划分（Single Stratified Random Split，简称 Single）。实验固定网络结构、优化器、损失函数、训练轮数、数据增强策略和独立测试集，仅改变训练集与验证集的划分协议。

CV 协议使用两个随机种子，每个种子将训练池分为 5 个互不重叠的折，并轮流将每一折作为验证集，共得到 10 次运行；Single 协议使用 10 个随机种子，每次按 80%/20% 进行一次分层划分，也得到 10 次运行。因此两种协议的运行次数相同，便于公平比较。

结果显示，两种协议在 AUC 和 AUPR 等概率级指标上的平均差异小于 0.32%，说明它们对模型整体排序能力的评估基本等价。**CV 在 F1 和 Precision 上略有优势，且 F1 的标准差更小；但 CV 得到的最优阈值波动更大**，标准差为 0.1789，而 Single 为 0.1147。两种协议相对于共同基准的泛化偏差均小于 0.008，未表现出明显的系统性高估或低估。

1.实验设计

1.1总体实验框架

实验采用 CIFAR-10 构造 Dog vs Non-Dog 不平衡二分类场景。两组实验使用完全相同的模型和训练配置，仅改变评估协议。

协议	数据划分方式	运行次数	主要目的
Repeated Stratified 5-Fold CV	种子 42、43；每个种子 5 折，每折轮流验证	10	学习并评估五折交叉验证
Single Stratified Random Split	种子 0~9；每次 80%/20% 分层随机划分	10	作为简单评估协议的对照

1.2 数据集构造

从 CIFAR-10 中保留以下 5 个原始类别，将 Dog 作为正类，其余 4 类合并为 Non-Dog 负类：

二分类类别	CIFAR-10 原始标签	训练+验证池	独立测试集
Dog（正类）	5	5,000	1,000
Non-Dog（负类）	2 Bird、3 Cat、4 Deer、7 Horse	20,000	4,000
合计	—	25,000	5,000

正类与负类比例为 1:4。25,000 张图像组成训练与验证池，5,000 张图像来自 CIFAR-10 原始测试集，作为所有运行共享的独立测试集。

独立测试集不参与训练、checkpoint 选择、学习率调整或阈值搜索。这样可以避免测试集信息泄漏，并保证 CV 与 Single 的最终测试结果具有可比性。

1.3五折交叉验证的具体流程

以某一个随机种子为例，CV 的流程如下：

- 1. 对 25,000 张训练+验证池样本进行分层随机打乱；
- 2. 将样本划分为 5 个折，每折约 5,000 张，并保持约 1:4 的类别比例；
- 3. 第 1 次运行使用第 1 折验证、第 2~5 折训练；
- 4. 第 2 次运行使用第 2 折验证、其余 4 折训练；
- 5. 依次完成第 3、4、5 折；
- 6. 对另一个随机种子重复上述过程；
- 7. 汇总 10 次独立运行在共同测试集上的指标，计算均值、标准差和指标分布。

因此，每个样本在同一个种子下恰好被用作一次验证样本、四次训练样本。五折交叉验证的优势是验证结果覆盖整个训练池；代价是需要训练 5 倍于单次划分的模型。

1.4单次分层随机划分流程

Single 协议在每个随机种子下将训练+验证池按 80%/20% 分层划分为训练集和验证集。每次只训练一个模型，验证集只占训练池的一个随机子集。使用 10 个种子重复后，对 10 次结果进行统计汇总。

1.5 模型架构

两种协议均使用同一个 CNN 分类器：

输入：3×32×32

└ Conv Block 1: Conv2d(3→256, 3×3, padding=1) + BN + ReLU + MaxPool(2×2)

| 输出：256×16×16

└ Conv Block 2: Conv2d(256→256, 3×3, padding=1) + BN + ReLU + MaxPool(2×2)

| 输出：256×8×8

└ Conv Block 3: Conv2d(256→256, 3×3, padding=1) + BN + ReLU

| 输出：256×8×8

└ Global Average Pooling: 256 维向量

└ 全连接头: Dropout(0.3) → Linear(256→512) + BN + ReLU

→ Dropout(0.2) → Linear(512→1)

输出：1 个 logit

1.6 训练配置与数据增强

参数	设置
优化器	Adam
初始学习率	4×10 ⁻³ ，含 2 个 epoch 的线性 warmup
权重衰减	1×10 ⁻⁴

参数	设置
损失函数	BCE With Logits Loss, pos_weight=4.0
批大小	256
最大训练轮数	30
学习率调度	Reduce LR On Plateau, 监控 Val-AUPR, patience=5, factor=0.5, min_lr=1×10 ⁻⁶
早停	patience=10, min_epochs=5, 监控 Val-AUPR
数值精度	FP16 AMP 混合精度

训练集使用 Random Horizontal Flip(p=0.5)、Random Crop(32, padding=4)、Random Rotation(15°) 和 Normalize；验证集、测试集只使用 Normalize。DataLoader 设置为 num_workers=4、pin_memory=True、persistent_workers=True、prefetch_factor=2。

1.7 模型选择与阈值搜索

训练过程中保存验证集 AUPR 最高的 checkpoint。训练结束后，仅在验证集上进行一次完整的阈值网格搜索，得到使 F1 最大的阈值 T^* ，再将该阈值固定并应用于独立测试集。

训练期间固定阈值 0.5 的 F1 仅作为参考，不进行反复阈值搜索。该设计可以减少验证集过拟合，并明确区分“模型选择”“阈值选择”和“最终测试评估”三个步骤。

1.8 评价指标

- **AUC**：衡量模型对正负样本排序的整体能力，不依赖固定分类阈值。
- **AUPR**：在类别不平衡场景下更关注正类识别质量，能够反映 Precision-Recall 权衡。
- **Precision**：预测为 Dog 的样本中真正为 Dog 的比例。
- **Recall**：所有 Dog 样本中被正确识别的比例。
- **F1 Score**：Precision 与 Recall 的调和平均，用于综合评价阈值决策效果。
- **最优阈值**：使验证集 F1 最大的概率阈值，用于观察决策边界的稳定性。

2. 实验结果

2.1 汇总统计

两种协议各运行 10 次，均在同一个独立测试集上进行评估。

```
=====
STATISTICAL SUMMARY
=====

--- CV (n=10) ---
    test_auc: 0.9239 ± 0.0065
    test_apr: 0.8089 ± 0.0144
    test_f1: 0.7170 ± 0.0129
    test_precision: 0.7389 ± 0.0409
    test_recall: 0.6993 ± 0.0333
    opt_threshold: 0.6605 ± 0.1789

--- Single (n=10) ---
    test_auc: 0.9225 ± 0.0063
    test_apr: 0.8063 ± 0.0158
    test_f1: 0.7098 ± 0.0167
    test_precision: 0.7234 ± 0.0382
    test_recall: 0.7000 ± 0.0414
    opt_threshold: 0.6768 ± 0.1147
```

指标	CV (均值 ± 标准差)	Single (均值 ± 标准差)	差值 CV-Single	相对差异
AUC	0.9239 ± 0.0065	0.9225 ± 0.0063	+0.0014	+0.15%
AUPR	0.8089 ± 0.0144	0.8063 ± 0.0158	+0.0026	+0.32%
F1 Score	0.7170 ± 0.0129	0.7098 ± 0.0167	+0.0072	+1.01%
Precision	0.7389 ± 0.0409	0.7234 ± 0.0382	+0.0155	+2.14%
Recall	0.6993 ± 0.0333	0.7000 ± 0.0414	-0.0007	-0.10%
最优阈值	0.6605 ± 0.1789	0.6768 ± 0.1147	-0.0163	-2.41%

主要发现如下：

- 1. AUC、AUPR 的差异均小于 0.32%，两种协议对模型排序能力的判断高度一致。
- 2. **CV 的平均 F1 比 Single 高 0.0072，且标准差更小，说明本实验中 CV 对 F1 的估计略稳定。**
- 3. Precision 的平均差异为 0.0155，是性能指标中相对较大的差异，但仍小于其运行间波动量级。
- 4. Recall 几乎相同，说明两种协议没有明显改变正类覆盖能力。

2.2 最佳单次运行

A	B	C	D	E	F	G	H	I	J	K	L	M
run_id	protocol	best_epoc	opt_thresl	val_f1_opt	test_loss	test_auc	testaupr	test_precision	test_recall	test_f1	seed	fold
1	CV	28	0.753618	0.697051	0.560015	0.923608	0.807725	0.760417	0.657	0.704936	42	1
2	CV	28	0.684673	0.736393	0.540288	0.934201	0.831859	0.78139	0.697	0.736786	42	2
3	CV	30	0.655126	0.747817	0.529764	0.935626	0.833858	0.743147	0.732	0.737531	42	3
4	CV	29	0.610804	0.729387	0.636523	0.918964	0.79972	0.762014	0.666	0.710779	42	4
5	CV	25	0.773317	0.7202	0.581736	0.919583	0.795981	0.707753	0.712	0.70987	42	5
6	CV	30	0.443367	0.74569	0.719978	0.920305	0.814158	0.777906	0.676	0.723381	43	1
7	CV	28	0.822563	0.728283	0.622773	0.921149	0.802476	0.717565	0.719	0.718282	43	2
8	CV	30	0.586181	0.727369	0.581924	0.927791	0.811885	0.764246	0.684	0.7219	43	3
9	CV	26	0.335025	0.711538	0.683358	0.918221	0.799707	0.646414	0.766	0.701144	43	4
10	CV	28	0.940754	0.726705	0.807825	0.919173	0.791672	0.728435	0.684	0.705518	43	5
11	Single	27	0.694523	0.712786	0.563614	0.923851	0.814699	0.693841	0.766	0.728137	0	0
12	Single	29	0.674824	0.735984	0.606566	0.918034	0.796782	0.708419	0.69	0.699088	1	0
13	Single	29	0.586181	0.718028	0.609526	0.918791	0.795254	0.723881	0.679	0.700722	2	0
14	Single	27	0.551709	0.725979	0.601003	0.924581	0.816598	0.73097	0.701	0.715671	3	0
15	Single	30	0.783166	0.75956	0.518342	0.937733	0.844349	0.796591	0.701	0.745745	4	0
16	Single	29	0.482764	0.716578	0.633423	0.918179	0.799057	0.685581	0.737	0.710361	5	0
17	Single	30	0.866884	0.722326	0.691461	0.920291	0.794479	0.673655	0.739	0.704816	6	0
18	Single	29	0.650201	0.717897	0.568091	0.921535	0.803817	0.704885	0.707	0.705941	7	0
19	Single	30	0.758543	0.734716	0.556848	0.925961	0.806222	0.756069	0.654	0.70134	8	0
20	Single	25	0.719146	0.717557	0.591538	0.915797	0.791906	0.759709	0.626	0.686404	9	0

协议	运行编号	Test AUC	Test AUPR	Test F1	最优阈值
CV	Run 03	0.9356	0.8327	0.7375	0.6923
Single	Run 15	0.9377	0.8381	0.7457	0.5385

Single 的最佳单次运行 F1 达到 0.7457，高于 CV 最佳运行的 0.7375。这说明单次划分在某些“幸运种子”下也可能取得很高的结果，因此不能只根据最佳一次运行判断评估协议优劣，应同时观察多次运行的均值和标准差。

2.3 泛化偏差

以全部 20 次运行的平均性能作为基准 $S_{\{benchmark\}}$ ，各协议偏差如下：

```
CV:
    test_auc: +0.0007
    test_aupr: +0.0013
    test_f1: +0.0036
test_precision: +0.0078
    test_recall: -0.0003
    opt_threshold: -0.0081

Single:
    test_auc: -0.0007
    test_aupr: -0.0013
    test_f1: -0.0036
test_precision: -0.0078
    test_recall: +0.0003
    opt_threshold: +0.0081
```

指标	CV 偏差	Single 偏差
AUC	+0.0007	-0.0007
AUPR	+0.0013	-0.0013
F1 Score	+0.0036	-0.0036
Precision	+0.0078	-0.0078
Recall	-0.0003	+0.0003
最优阈值	-0.0081	+0.0081

CV 整体呈轻微正偏差，Single 呈轻微负偏差，但最大绝对偏差仅为 0.0078，远小于多数指标的标准差。

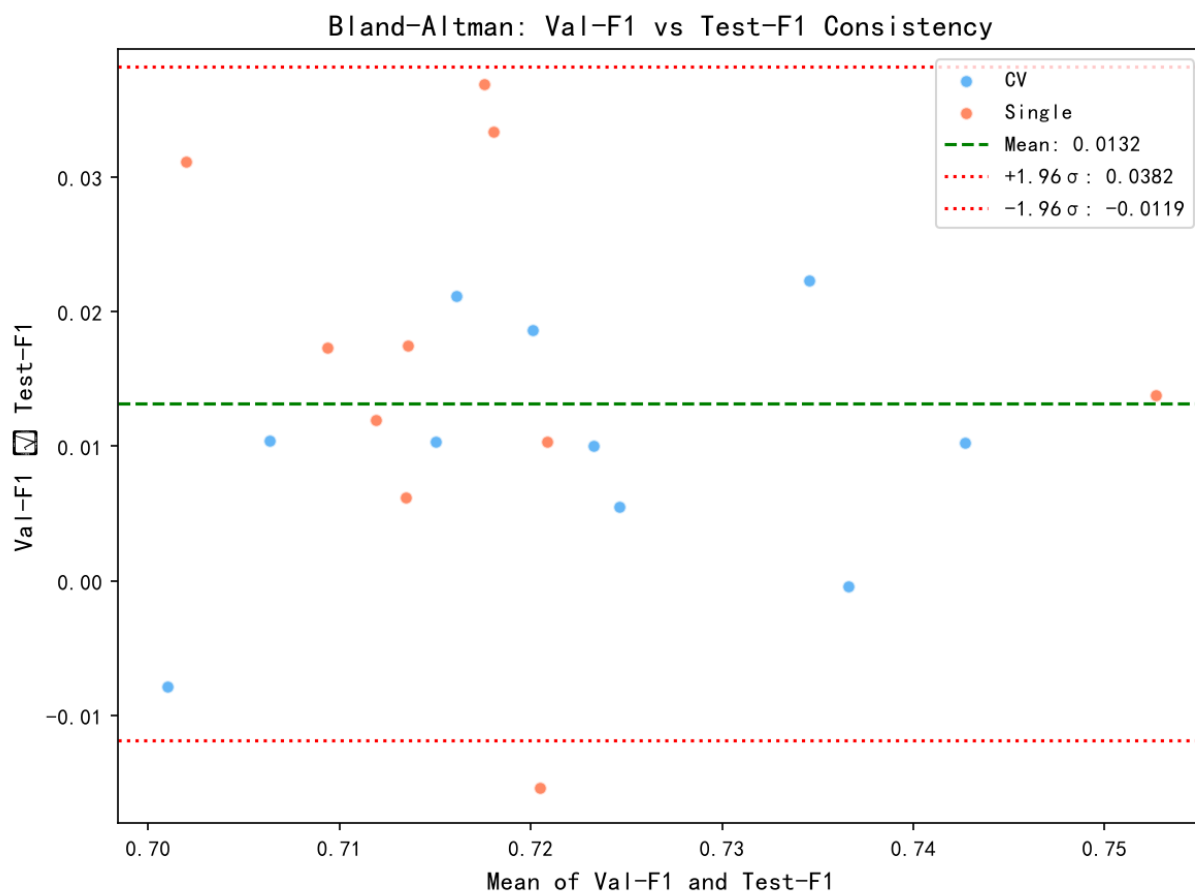
2.4 阈值稳定性

协议	均值	标准差	变异系数	范围
CV	0.6605	0.1789	27.09%	0.44~0.94
Single	0.6768	0.1147	16.95%	0.54~0.86

CV 最优阈值的标准差约为 Single 的 1.56 倍，取值范围也明显更宽。由此可见，五折交叉验证可以更充分地暴露不同数据子集带来的不确定性，但不一定能让最终部署阈值更稳定。

2.5 验证集与测试集 F1 的一致性

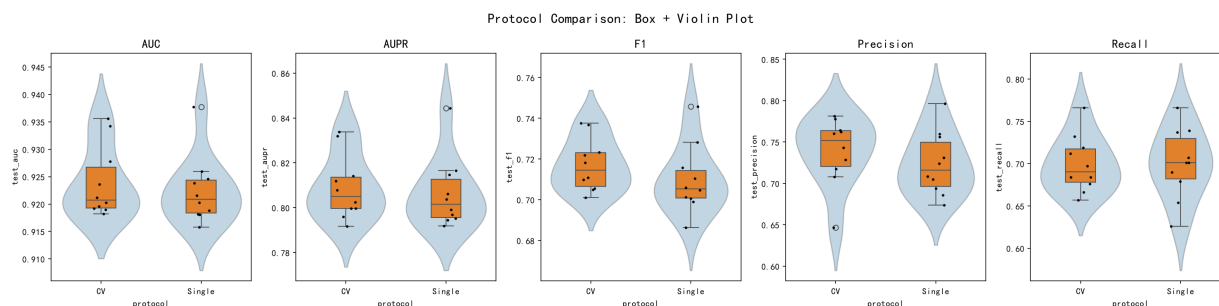
采用 Bland-Altman 分析比较验证集 F1 与测试集 F1:



- 均值偏差为 +0.0132, 即验证集 F1 平均比测试集 F1 高约 0.013;
- 95% 一致性界限为 [-0.0119, +0.0382];
- 绝大多数点位于一致性界限内, 说明验证集 F1 对测试集 F1 有一定预测参考价值;

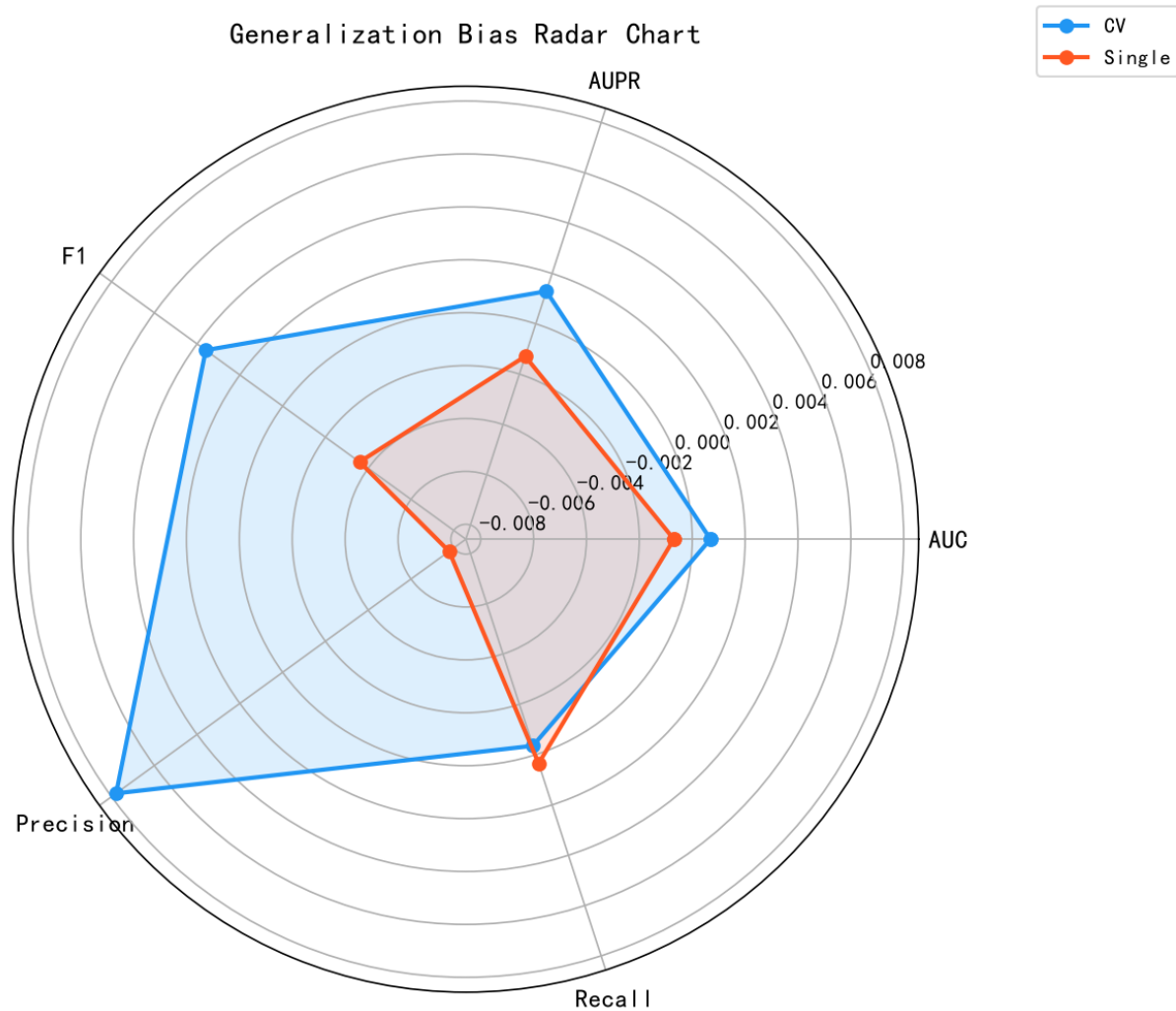
3. 可视化结果分析

3.1 指标分布图



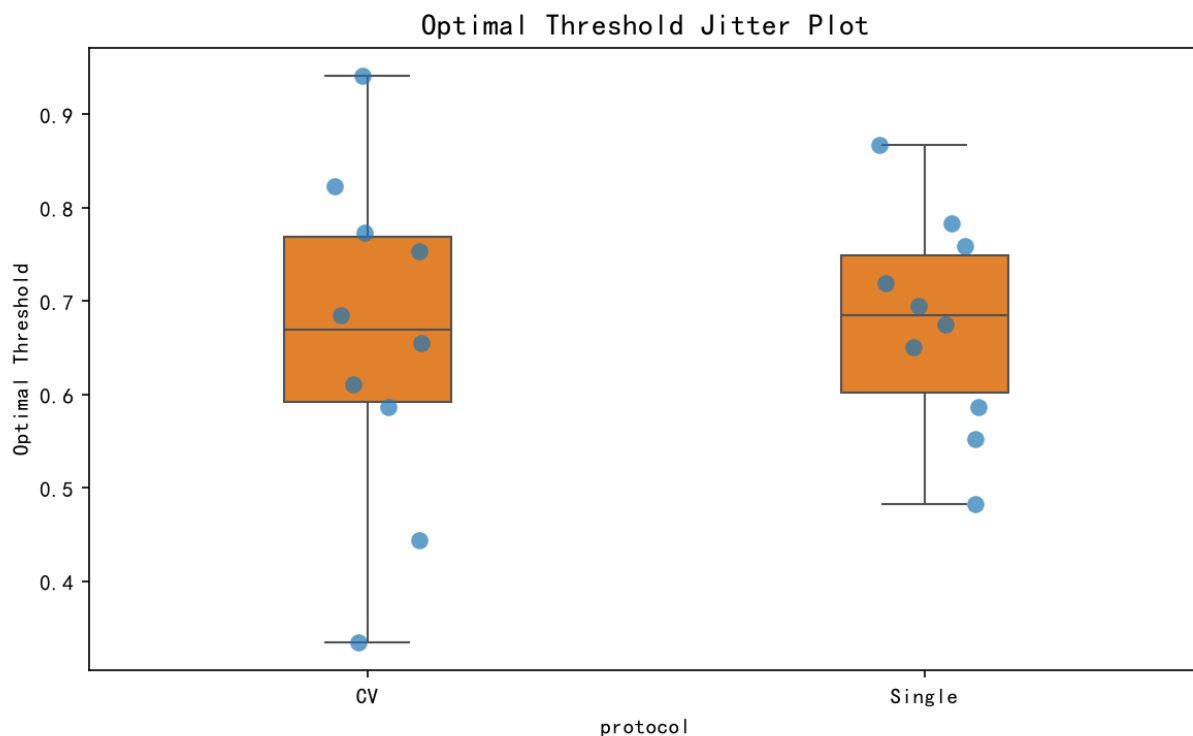
- AUC 和 AUPR 的两组分布高度重叠;
- CV 的 F1 中位数略高、箱体略紧, 支持其 F1 估计更稳定的结论;
- Precision 中 CV 的分布整体偏高;
- Recall 两者几乎重合。

3.2 泛化偏差雷达图



CV 在 F1 和 Precision 方向表现为轻微正偏差，Single 的轮廓大致呈对称的负偏差。AUC、AUPR 和 Recall 方向都接近中心，说明两种协议在这些指标上的偏差很小。

3.3 阈值抖动图

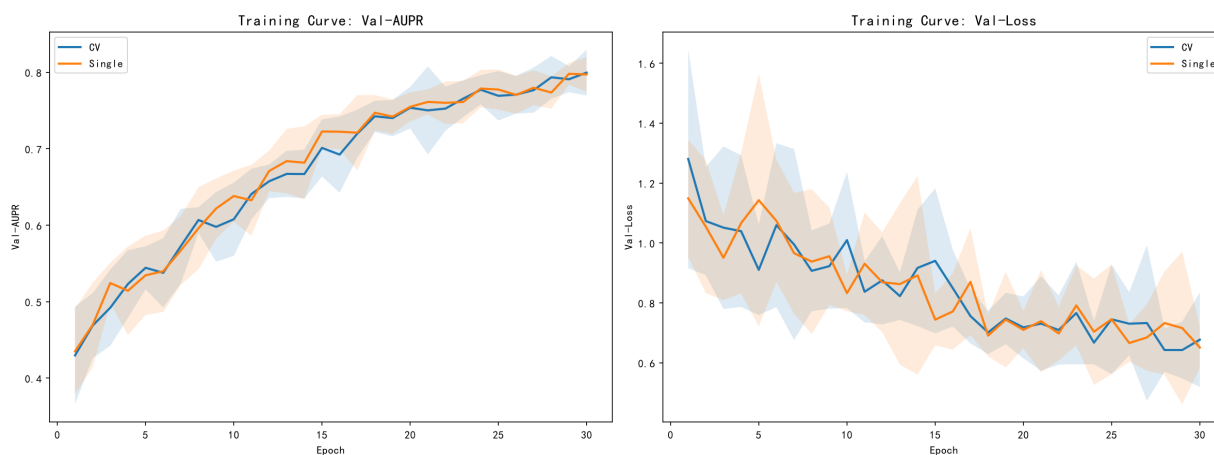


CV 的最优阈值散布约为 0.35~0.95，Single 约为 0.50~0.85。

该图最直观地体现了两种协议的差异：

- **CV 更能揭示数据分区导致的阈值不确定性**，而 Single 的重复验证集存在较高样本重叠，因此表面上更集中。

3.4 训练曲线



两种协议的 Val-AUPR 和 Val-Loss 均表现出相似的收敛过程：Val-AUPR 从约 0.65 上升至约 0.80，Loss 从约 1.3 降至 0.6~0.7，并在训练后期趋于平稳。这说明评估协议没有明显改变模型的训练动态，差异主要来自数据划分和最终统计方式。

4.现象分析

4.1 AUC 和 AUPR 差异很小

AUC 和 AUPR 主要评价模型输出分数的排序能力，不依赖某一个固定阈值。只要不同训练运行学到的整体排序关系相近，数据划分协议对这两个指标的影响就可能较小。本实验的结果说明，在 25,000 张训练+验证样本的规模下，CV 和 Single 对概率级性能的判断基本一致。

4.2 CV 的 F1 略高

五折 CV 中，每个样本都曾经作为验证样本，验证过程覆盖了完整的数据分布。相比只观察某一个 20% 子集，CV 的模型选择信息更充分，因此 F1 和 Precision 可能获得轻微提升。不过，F1 的平均差异只有 0.0072，尚不能据此断言 CV 在所有类似任务中都显著优于 Single；若要进行严格判断，还应进行配对统计检验并报告置信区间。

4.3 CV 的阈值反而更不稳定

五个验证折虽然类别比例相近，但图像难度、背景和样本特征分布仍可能不同。每折的 F1-阈值曲线形状不同，就会导致最优阈值在不同折之间明显变化。Single 的 10 个验证集来自同一数据池且相互重叠率较高，期望重叠率约为 64%，因此阈值可能表现得更集中，但这种集中并不等于更充分的独立证据。

结果的“稳定”与不确定性的“真实”不是完全相同的概念。

- Single 可能给出更稳定的部署阈值，**CV 则可能更真实地展示不同数据子集对阈值选择的影响。**

4.4 五折交叉验证的优点与代价

优点包括：

- 每个样本都参与训练和验证，数据利用率高；
- 能观察不同验证子集带来的结果波动；
- 对小样本任务通常比单次划分更可靠；
- 可据此计算性能均值、标准差和置信区间。

代价包括：

- 同一随机种子需要训练 5 个模型，计算成本约为单次划分的 5 倍；
- 不同折的阈值或 checkpoint 可能差异较大；
- 若在一折反复调参，仍可能产生验证集过拟合；
- 普通 K 折 CV 并不能替代真正独立的测试集。

5. 结论与实践建议

5.1 主要结论

1. 在本 CIFAR-10 不平衡二分类任务中，CV 与 Single 对 AUC、AUPR 的估计基本等价。
2. CV 的平均 F1 和 Precision 略高，且 F1 的运行间标准差更小。
3. Single 的最优阈值明显更稳定；CV 的宽阈值分布揭示了更大的数据划分不确定性。
4. 两种协议的泛化偏差均很小，未发现明显的系统性估计偏差。
5. 评估协议主要影响最终统计结果和阈值不确定性，对训练收敛曲线的影响不明显。

附录：重复分层五折代码

```
# 遍历交叉验证的随机种子列表（如 [42, 43]），实现重复分层k折交叉验证
for seed in Config.CV_SEEDS:
```

```
    rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=1, random_state=seed)
```

- Repeated Stratified K Fold #重复分层K折分割器
- n_splits = 5 #数据分为5折
- n_repeats=1 #每个种子仅执行1轮完整的5折划分（通过外层seed循环实现"重复"）
- random_state=seed #固定当前种子的随机性，确保该种子下的5折划分可复现

```
    for fold_idx, (train_idx, val_idx) in enumerate(rskf.split(pool_idx, pool_labels)):
```

获取

- fold_idx #折编号
- train_idx #训练集索引
- val_idx #验证集索引

in enumerate(元素, 索引) #将对象组合为序列，获取

- rskf.split () #分层划分
- pool_idx #池数据
- pool_labels #池标签

rskf.split(pool_idx, pool_labels) #对池数据按池标签进行分层划分，保证保证每折正负样本比例一致

```

# 根据当前折的索引构建训练集数据集对象，应用训练集数据增强(transform_train)
train_ds = BinaryCIFAR(full_train, pool_idx[train_idx], pool_labels[train_idx], transform_train)
# 根据当前折的索引构建验证集数据集对象，应用测试集变换(transform_test, 无增强)
val_ds = BinaryCIFAR(full_train, pool_idx[val_idx], pool_labels[val_idx], transform_test)
# 执行单次完整实验流程（训练→早停→阈值搜索→测试集评估），返回指标字典
row = run_single_experiment(run_counter, total_runs, train_ds, val_ds, test_dataset, all_histories)

# 调用单次实验执行函数，完成当前折/划分的完整训练与评估流程

row = run_single_experiment ()

#返回值row是一个字典，包含：best_epoch、opt_threshold、val_f1_opt、test_loss/auc/aupr/precision/recall/f1等指标

run_counter, # 全局运行编号（从0开始递增），用于日志打印时标识当前是第几次实验

total_runs, # 总实验次数（CV折数 + Single划分数 = 20），与run_counter配合显示进度

train_ds, # 当前折/划分的训练集Dataset对象（已应用数据增强transform_train)

val_ds, # 当前折/划分的验证集Dataset对象（无数据增强，仅做标准化transform_test)

test_dataset, # 全局固定测试集Dataset对象（所有20次实验共用同一测试集，保证评估基准一致)

all_histories # 列表引用（mutable），函数内部会将当前run的逐epoch训练历史append进去，实现跨run的历史记录汇总

row['protocol'] = 'cv'# 标记评估协议类型为交叉验证
row['seed'] = seed# 记录当前使用的随机种子
row['fold'] = fold_idx + 1 # 记录当前折号（从1开始计数，符合人类阅读习惯）

# 将本次运行结果追加到总结果列表中
all_results.append(row)
# 全局运行计数器自增，用于日志打印和进度追踪
run_counter += 1

```